

Базы данных

Тема 12

Транзакции.

Управление параллельной обработкой в БД

- управление параллельной обработкой (concurrency control) направлено на то, чтобы исключить *непредусмотренное* влияние действий одного пользователя на действия другого;
- управление параллельной обработкой, как правило, предполагает компромисс между уровнем изоляции различных операций друг от друга и производительностью системы;
- транзакция (transaction) является последовательностью операций, выполненных как одна логическая единица работы (logical unit of work), т.е. либо все действия внутри транзакции выполняются успешно, либо не выполняется ни одно из них:
 - BEGIN TRANSACTION;
 - COMMIT | ROLLBACK TRANSACTION.

Параллельная обработка транзакций

- параллельные транзакции (parallel transactions) – две или более транзакций, обрабатываемые одновременно;
- проблемы параллельной обработки:
 - *грязное / несогласованное чтение* (dirty / inconsistent reads);
 - *невоспроизводимое / фантомное чтение* (unrepeatable / phantom reads);
 - *потерянное обновление* (lost / concurrent update);
- необходимо упорядочить операции внутри транзакций таким образом, чтобы предотвратить *нежелательное влияние* одной транзакции на другую

Проблема грязного / несогласованного чтения

- грязное чтение:

Write-Read Conflict

T_1 : WRITE(A)

T_1 : ABORT

T_2 : READ(A)

- несогласованное чтение:

Write-Read Conflict

T_1 : A := 20; B := 20;

T_1 : WRITE(A)

T_1 : WRITE(B)

T_2 : READ(A);

T_2 : READ(B);

Проблема невоспроизводимого чтения

Read-Write Conflict

T_1 : WRITE(A)

T_2 : READ(A);

T_2 : READ(A);

Проблема потерянного обновления

User A

1. Read item 100
(item count is 10).
2. Reduce count of items by 5.
3. Write item 100.

User B

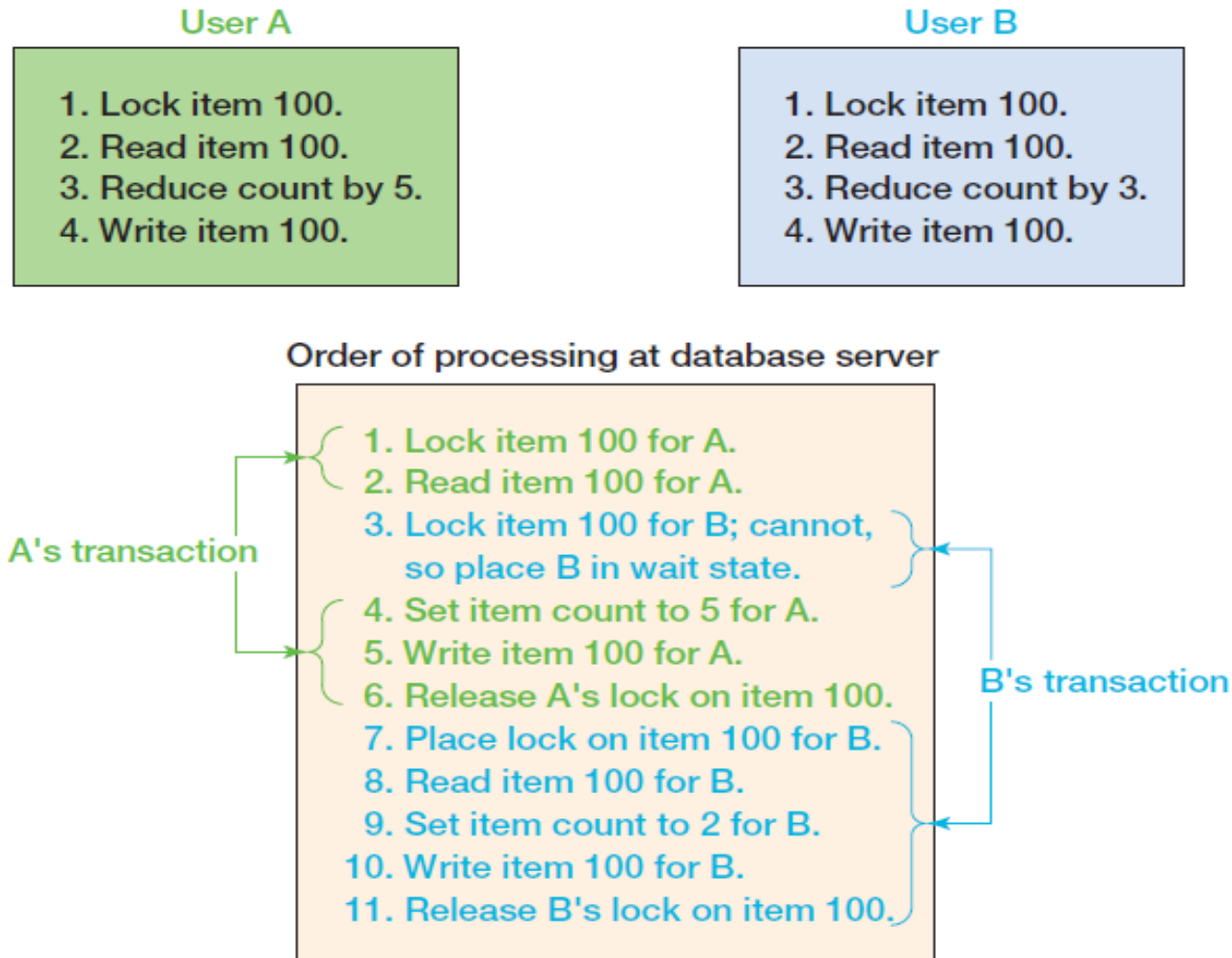
1. Read item 100
(item count is 10).
2. Reduce count of items by 3.
3. Write item 100.

Order of processing at database server

1. Read item 100 (for A).
2. Read item 100 (for B).
3. Set item count to 5 (for A).
4. Write item 100 for A.
5. Set item count to 7 (for B).
6. Write item 100 for B.

Note: The change and write in steps 3 and 4 are lost.

Решение проблемы потерянного обновления с помощью транзакций



Свойства транзакций

- транзакция может обладать четырьмя свойствами:
 - *атомарность (atomicity)*: транзакция должна быть атомарной единицей работы; должны быть выполнены либо все входящие в нее модификации данных, либо ни одно из них не должно быть выполнено;
 - *согласованность (consistency)*: по завершении транзакция должна оставить все данные в согласованном состоянии (включая все внутренние структуры данных), для этого модификации должны применяться к данным в том состоянии, в котором они были на момент начала операции:
 - на уровне оператора (statement level consistency);
 - на уровне транзакции (transaction level consistency);
 - *изоляция (isolation)*: выполняемые транзакцией модификации должны быть (могут быть) изолированы от любых модификаций, проводимых другими транзакциями;
 - *устойчивость (durability)*: после завершения транзакции произведенные ею действия фиксируются в системе (даже в случае системного сбоя).

Согласованность транзакций

- *согласованность (consistency)*: по завершении транзакция должна оставить все данные в согласованном состоянии (включая все внутренние структуры данных), для этого модификации должны применяться к данным в том состоянии, в котором они были на момент начала операции:
 - на уровне оператора (statement level consistency);
 - на уровне транзакции (transaction level consistency).

```
BEGIN TRANSACTION;  
UPDATE CUSTOMER  
    SET AreaCode = '425'  
    WHERE ZipCode = '98050';  
COMMIT TRANSACTION;
```

```
BEGIN TRANSACTION;  
  
UPDATE CUSTOMER  
    SET AreaCode = '425'  
    WHERE ZipCode = '98050';
```

```
UPDATE CUSTOMER  
    SET Discount = 0.05  
    WHERE AreaCode = '425';  
  
COMMIT TRANSACTION;
```

Изоляция транзакций

- проблемы изоляции транзакций:
 - «грязное» чтение (*dirty read*) – чтение транзакцией записи, измененной другой транзакцией, при этом эти изменения еще не зафиксированы;
 - невозпроизводимое чтение (*non-repeatable read*) – при повторном чтении транзакция обнаруживает измененные или удаленные данные, зафиксированные другой транзакцией;
 - фантомное чтение (*phantom read*) – при повторном чтении транзакция обнаруживает новые строки, вставленные другой завершенной транзакцией;
- изоляция транзакций означает, что клиент СУБД может устанавливать, а сама СУБД, соответственно, обеспечивает управление блокировками таким образом, чтобы достичь нужного уровня изоляции.

Уровни изоляции транзакций

- стандарт SQL92 определяет четыре уровня изоляции транзакций:
 - незавершенное чтение (read uncommitted);
 - завершенное чтение (read committed);
 - воспроизводимое чтение (repeatable read);
 - сериализуемость (serializable);
- уровень изоляции влияет на производительность базы данных;
- уровни изоляции косвенно связаны с типами курсоров (однонаправленные, статические, динамические и основанные на потенциальном ключе).

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Сериализуемость транзакций

- сериализуемость транзакций (serializable transactions) – такая схема обработки транзакций, при которой результат их параллельной обработки такой же, как если бы они выполнялись последовательно;
- стратегии, обеспечивающие изоляцию транзакций:
 - пессимистическая блокировка (pessimistic locking): блокировка ресурсов (resource locking), т.е. предотвращение чтения тех записей, которые модифицируются другой транзакцией;
 - оптимистическая блокировка (optimistic locking):
 - OCC (optimistic concurrency control);
 - TS-based (timestamp-based);
 - MVCC (multiversion concurrency control);

Блокировка ресурсов

- инициатор блокировки:
 - СУБД: *неявная блокировка (implicit lock)*;
 - программа / запрос: *явная блокировка (explicit lock)*;
- степень (глубина) детализации блокировки (*lock granularity*):
 - на уровне базы данных;
 - на уровне таблицы;
 - на уровне страницы;
 - на уровне строки;
- режим (тип) блокировки:
 - *монопольный (exclusive lock)* : блокируются все виды доступа к элементу;
 - *разделяемый (shared lock)* : блокируется только запись.

Двухфазная блокировка

- один из способов достижения сериализуемости – использование *двухфазной блокировки* (two-phase locking):
 - *фаза нарастания* (growing phase): транзакция может налагать блокировки по мере необходимости, до тех пор, пока не снимается хотя бы одна блокировка;
 - *фаза сжатия* (shrinking phase): после снятия одной из блокировок никакие другие накладываться уже не могут;
- модификация двухфазной блокировки: снятие блокировок непосредственно в момент фиксации / отката транзакции (предотвращает каскадные откаты).

Взаимная блокировка (deadlock, deadly embrace)

- предотвращение взаимной блокировки:
 - блокирование всех ресурсов одновременно в начале операции;
 - требование блокировки в одинаковом порядке;
- обнаружение и устранение взаимной блокировки: откат одной из транзакций, вызвавших блокировку (deadlock breaking).

User A

1. Lock paper.
2. Take paper.
3. Lock pencils.

User B

1. Lock pencils.
2. Take pencils.
3. Lock paper.

Order of processing at database server

1. Lock paper for user A.
2. Lock pencils for user B.
3. Process A's requests; write paper record.
4. Process B's requests; write pencil record.
5. Put A in wait state for pencils.
6. Put B in wait state for paper.

**** Locked ****

Оптимистическая блокировка

- T_i - транзакция, имеет уникальную временную метку $TS(T_i)$ (таким образом, все транзакции строго упорядочены);
- X - объект, операции чтения/записи над которым также имеют временную метку $RTS(X)$ / $WTS(X)$ или фактическое время выполнения $RT(X)$ / $WT(X)$
- OCC (optimistic concurrency control):

$$\exists X_k \mid WT(X_k \mid T_j \neq T_i) > TS(T_i) \Rightarrow \text{cancel } T_i$$

Оптимистическая блокировка (2)

- TS-based (timestamp-based);
 - при чтении:
$$WTS(X_k) > TS(T_i) \Rightarrow \text{cancel } T_i$$
$$RTS(X_k) \leftarrow \max(RTS(X_k), TS(T_i))$$
 - при записи:
$$RTS(X_k) > TS(T_i) \Rightarrow \text{cancel } T_i$$
$$WTS(X_k) > TS(T_i) \Rightarrow \text{skip } T_i$$
$$WTS(X_k) \leftarrow TS(T_i)$$
 - в ряде случаев необходимо ожидание завершения взаимосвязанных транзакций;
- MVCC (multiversion concurrency control):
 - множественные версии объекта («снимки»);
 - при чтении транзакция получает самую последнюю версию объекта *на момент начала данной транзакции*;
 - при записи:
$$TS(T_i) < RTS(X_k) = TS(T_j) \Rightarrow \text{cancel } T_i$$
$$X_k^t \rightarrow X_k^{t+1}; RTS(X_k^{t+1}) = WTS(X_k^{t+1}) = TS(T_i)$$
 - условие удаления «устаревших» версий:
$$\exists X_k^{t+1} \mid TS(X_k^{t+1}) > TS(X_k^t) \ \& \ \forall T_i \ TS(T_i) > TS(X_k^{t+1})$$

Восстановление баз данных

- повторная обработка (reprocessing):
 - периодические снимки базы данных и записи о всех операциях со времени последнего сохранения;
 - нереализуемый подход в силу отсутствия времени и асинхронного характера первоначальных изменений;
- откат-накат (rollback-rollforward):
 - периодические снимки базы данных и журнал транзакций со времени последнего сохранения, содержащий все изменения в базе данных в хронологическом порядке;
 - rollforward: восстановление базы данных и выполнение всех корректных транзакций;
 - rollback: отмена изменений, вызванных некорректными или незавершенными транзакциями, и выполнение всех корректных транзакций, выполнявшихся в момент сбоя;
 - для отмены транзакции журнал должен содержать копию записей базы данных до их изменения (исходные образы, *before-images*);
 - для повторного выполнения транзакции журнал должен содержать копию записей базы данных после их изменения (конечные образы, *after-images*).

Управление параллельной обработкой в SQL Server

- определение уровня изоляции транзакции:

```
SET TRANSACTION ISOLATION LEVEL
    { READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ
      | SNAPSHOT | SERIALIZABLE }
```

- определение характеристик курсора:

```
DECLARE cursor_name CURSOR
    [ LOCAL | GLOBAL ]
    [ FORWARD_ONLY | SCROLL ]
    [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
    [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
    [ TYPE_WARNING ]
    FOR select_statement
    [ FOR UPDATE [ OF column_name [ , ...n ] ] ]
```

- блокировочные подсказки инструкций (SELECT, INSERT, UPDATE, DELETE):

```
SELECT ... FROM ... WITH
```

Определение уровня изоляции транзакций

- возможные уровни изоляции:
 - READ UNCOMMITTED;
 - **READ COMMITTED**;
 - READ_COMMITTED_SNAPSHOT (ON | **OFF**);
 - REPEATABLE READ;
 - SNAPSHOT – согласованность на уровне транзакции;
 - SERIALIZABLE;
- возможно переключение уровня изоляции внутри транзакции (кроме → SNAPSHOT, правила блокировок применяются на основании текущего уровня изоляции);
- при выходе из хранимой процедуры / триггера уровень изоляции восстанавливается до уровня, актуального на момент вызова.

		Isolation Level			
		Read Uncommitted	Read Committed	Repeatable Read	Serializable
Problem Type	Dirty Read	Possible	Not Possible	Not Possible	Not Possible
	Nonrepeatable Read	Possible	Possible	Not Possible	Not Possible
	Phantom Read	Possible	Possible	Possible	Not Possible

Уровни изоляции транзакций (пример)

- разделяемые блокировки удерживаются все время до завершения транзакции

```
USE AdventureWorks;  
GO  
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
GO  
BEGIN TRANSACTION;  
GO  
SELECT * FROM HumanResources.EmployeePayHistory;  
GO  
SELECT * FROM HumanResources.Department;  
GO  
COMMIT TRANSACTION;  
GO
```

Блокировочные подсказки инструкций

SELECT/UPDATE/INSERT/DELETE ... WITH (...)

- HOLDLOCK (= SERIALIZABLE)
- NOLOCK (= READUNCOMMITTED)
- READCOMMITTED
- READCOMMITTEDLOCK
- READPAST (пропуск заблокированных другими транзакциями записей)
- READUNCOMMITTED
- REPEATABLEREAD
- SERIALIZABLE
- PAGLOCK | ROWLOCK | TABLOCK
- TABLOCKX
- XLOCK

```
UPDATE Production.Product
WITH (TABLOCK)
SET ListPrice = ListPrice * 1.10
WHERE ProductNumber LIKE 'BK-%'
```

Блокировочные подсказки инструкций (пример)

```
USE AdventureWorks;
GO
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
GO
BEGIN TRANSACTION;
GO
SELECT Title FROM HumanResources.Employee WITH (NOLOCK);
GO
-- Get information about the locks held by the
   transaction.
SELECT resource_type, resource_subtype, request_mode FROM
   sys.dm_tran_locks WHERE request_session_id = @@spid;
-- End the transaction.
ROLLBACK;
GO
```

Определение характеристик курсора

```
DECLARE cursor_name CURSOR
  [ LOCAL | GLOBAL ]
  [ FORWARD ONLY | SCROLL ]
  [ STATIC | KEYSET | DYNAMIC | FAST_FORWARD ]
  [ READ_ONLY | SCROLL_LOCKS | OPTIMISTIC ]
  [ TYPE_WARNING ]
  FOR select statement
  [ FOR UPDATE [ OF column_name [ , ...n ] ] ]
```

- **FORWARD_ONLY | SCROLL** – возможность прокрутки курсора;
- **STATIC | KEYSET | DYNAMIC | FAST_FORWARD** – тип курсора;
- **READ_ONLY | SCROLL_LOCKS | OPTIMISTIC (WITH VALUES / WITH ROW VERSIONING)** – управление изменениями и блокировками записей;
- возможно неявное изменение типа курсора, если формирующая его инструкция выборки содержит элементы, не соответствующие требованиям курсора.

Режимы транзакций

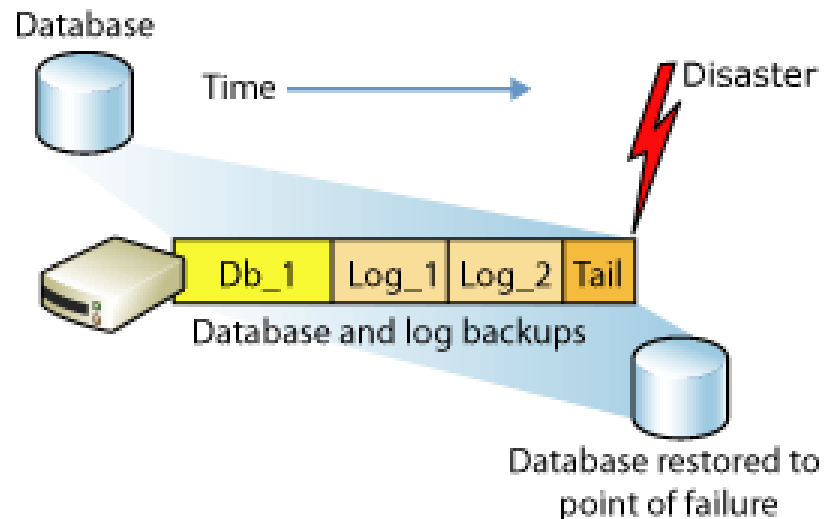
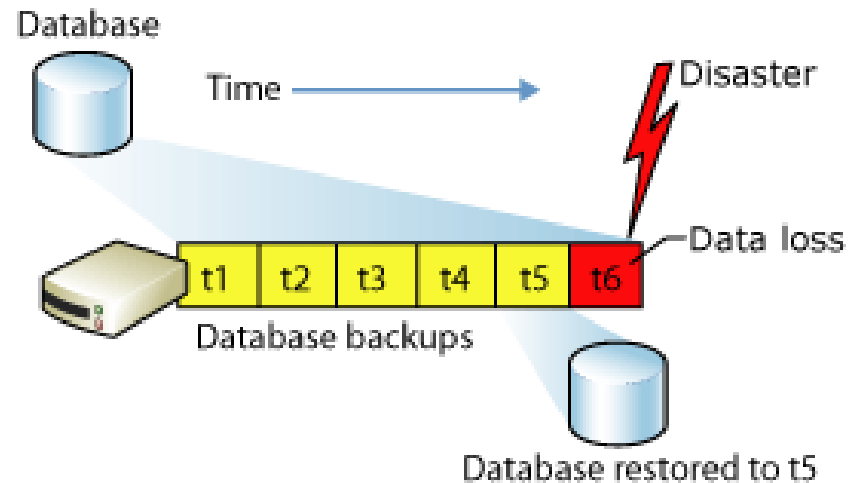
- явные транзакции:
 - запускаются посредством инструкции BEGIN TRANSACTION;
 - завершение транзакции осуществляется с использованием инструкций COMMIT TRANSACTION / WORK (фиксация) или ROLLBACK TRANSACTION / WORK (откат);
 - управление транзакциями также может осуществляться через функции API – ADO, ODBC, OLEDB (ITransactionLocal::StartTransaction(), ITransaction::Commit() и ITransaction::Abort());
- автоматически фиксируемые транзакции (режим по умолчанию):
 - каждая отдельная инструкция фиксируется после завершения;
- неявные транзакции:
 - режим запускается через инструкцию SET IMPLICIT_TRANSACTIONS ON или через соответствующие функции API (ODBC, OLEDB);
 - очередная инструкция ALTER TABLE, INSERT, CREATE, OPEN, DELETE, REVOKE, DROP, SELECT, FETCH, TRUNCATE TABLE, GRANT, UPDATE автоматически запускает новую транзакцию;
 - необходимо только фиксировать или выполнять откат каждой транзакции (COMMIT / ROLLBACK), после завершения транзакции следующая инструкция (из перечисленного списка) запускает новую транзакцию.

Неявные транзакции: пример (T-SQL)

```
CREATE TABLE ImplicitTran
    (Cola int PRIMARY KEY,
     Colb char(3) NOT NULL);
GO
SET IMPLICIT_TRANSACTIONS ON;
GO
-- First implicit transaction started by an INSERT statement.
INSERT INTO ImplicitTran VALUES (1, 'aaa');
GO
INSERT INTO ImplicitTran VALUES (2, 'bbb');
GO
-- Commit first transaction.
COMMIT TRANSACTION;
GO
-- Second implicit transaction started by a SELECT statement.
SELECT COUNT(*) FROM ImplicitTran;
GO
INSERT INTO ImplicitTran VALUES (3, 'ccc');
GO
SELECT * FROM ImplicitTran;
GO
-- Commit second transaction.
COMMIT TRANSACTION;
GO
SET IMPLICIT_TRANSACTIONS OFF;
GO
```

Восстановление

- создание резервной копии (данных и журнала транзакций):
 - полные (full backup);
 - частичные (differential backup);
- модели ВОССТАНОВЛЕНИЯ:
 - простая (simple recovery model);
 - полная (full recovery model);
 - частичная (bulk-logged recovery model).



Лабораторная работа №10. Режимы выполнения транзакций

1. Исследовать и проиллюстрировать на примерах различные уровни изоляции транзакций MS SQL Server, устанавливаемые с использованием инструкции **SET TRANSACTION ISOLATION LEVEL**
2. Накладываемые блокировки исследовать с использованием **sys.dm_tran_locks**

Лабораторная работа №11.

Создание базы данных и запросов к ней в СУБД SQL Server 2012

1. создать базу данных, спроектированную в рамках лабораторной работы №4, используя изученные в лабораторных работах 5-10 средства SQL Server 2012:
 - поддержания создания и физической организации базы данных;
 - различных категорий целостности;
 - представления и индексы;
 - хранимые процедуры, функции и триггеры;
2. создание объектов базы данных должно осуществляться средствами DDL (CREATE/ALTER/DROP), в обязательном порядке иллюстрирующих следующие аспекты:
 - добавление и изменение полей;
 - назначение типов данных;
 - назначение ограничений целостности (PRIMARY KEY, NULL/NOT NULL/UNIQUE, CHECK и т.п.);
 - определение значений по умолчанию;

Лабораторная работа №11

(продолжение)

3. в рассматриваемой базе данных должны быть тем или иным образом (в рамках объектов базы данных или дополнительно) созданы запросы DML для:
 - выборки записей (команда SELECT);
 - добавления новых записей (команда INSERT), как с помощью непосредственного указания значений, так и с помощью команды SELECT;
 - модификации записей (команда UPDATE);
 - удаления записей (команда DELETE);
4. запросы, созданные в рамках пп.2,3 должны иллюстрировать следующие возможности языка:
 - удаление повторяющихся записей (DISTINCT);
 - выбор, упорядочивание и именование полей (создание псевдонимов для полей и таблиц / представлений);
 - соединение таблиц (INNER JOIN / LEFT JOIN / RIGHT JOIN / FULL OUTER JOIN);
 - условия выбора записей (в том числе, условия NULL / LIKE / BETWEEN / IN / EXISTS);
 - сортировка записей (ORDER BY - ASC, DESC);
 - группировка записей (GROUP BY + HAVING, использование функций агрегирования – COUNT / AVG / SUM / MIN / MAX);
 - объединение результатов нескольких запросов (UNION / UNION ALL / EXCEPT / INTERSECT);
 - вложенные запросы.

Вопросы к экзамену

- Управление параллельной обработкой в БД. Транзакции. Блокировка и взаимная блокировка.
- Транзакции: свойства, уровни изоляции. Восстановление баз данных.
- Управление параллельной обработкой в MS SQL Server.
- Транзакции в SQL Server. Режимы транзакций.
- Восстановление баз данных. Восстановление в MS SQL Server 2012.